

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Application-Based Problem-Solving

Course Code:	CSA0901	Course Title:	Programming in Java
CO Assessed:	CO5 – Precision (BL3,BL4)	Assessment:	ASSESSMENT TOOL 2 - Application-Based Problem-Solving
Weightage:	40%	Total Marks:	100
CO1 Statement:	Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)		
Student Name:		Reg. No.:	
Section Batch: /	SLOT B	Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 4 questions Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 30 minutes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Application of Concepts	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Analytical & Decision-Making Skills	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Solution Design & Approach	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Clarity, Justification & Presentation	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Secure Employee Record Management System

Develop a Java application for an organization to securely manage employee records using **hashing and an appropriate hash function** for employee identification. Store employee details in a file using **byte/character streams** and implement serialization for storing employee objects. Provide options to add, search, update, and delete employee records. The system should handle duplicate IDs and invalid file operations appropriately.

2. Bank Transaction File and Database System

Design a banking application that maintains customer and transaction information using **Java I/O and JDBC**. Store transaction logs using stream I/O and serialize customer objects for backup. Establish JDBC connectivity with a relational database and implement **INSERT, UPDATE, DELETE, and SELECT** operations using appropriate Statement types. The application must validate transactions and handle database and I/O exceptions.

1. Secure Employee Record Management System

Aim

To develop a Java application that manages employee records using **hashing, file handling, byte/character streams, and serialization**, with options to add, search, update, and delete employee records.

Program

```
import java.io.*;import java.util.*;
class Employee implements Serializable {

    private static final long serialVersionUID = 1L;

    int id;
    String name;
    String department;
    double salary;

    Employee(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public String toString() {
        return "ID: " + id +
            "\nName: " + name +
            "\nDepartment: " + department +
            "\nSalary: " + salary;
    }
}

class EmployeeSystem {
```

```

static final String FILE = "employees.dat";

// Hash function
static int hashFunction(int id) {
    return id % 10;
}

// Read employee records
static ArrayList<Employee> readEmployees() {

    ArrayList<Employee> employees = new ArrayList<>();

    try {
        ObjectInputStream in =
            new ObjectInputStream(
                new FileInputStream(FILE));

        employees =
            (ArrayList<Employee>) in.readObject();

        in.close();
    }

    catch (FileNotFoundException e) {
        // File will be created when first record is added
    }

    catch (IOException | ClassNotFoundException e) {
        System.out.println(
            "Error reading employee file.");
    }

    return employees;
}

// Write employee records
static void writeEmployees(
    ArrayList<Employee> employees) {

    try {

        ObjectOutputStream out =
            new ObjectOutputStream(
                new FileOutputStream(FILE));

        out.writeObject(employees);
        out.close();

    }

    catch (IOException e) {

        System.out.println(
            "Error writing employee file.");
    }

}

// Add employee
static void addEmployee(Employee emp) {

```

```

        ArrayList<Employee> employees =
            readEmployees();

        for (Employee e : employees) {

            if (e.id == emp.id) {

                System.out.println(
                    "Error: Duplicate Employee ID.");
                return;
            }
        }

        employees.add(emp);
        writeEmployees(employees);

        System.out.println(
            "Employee added successfully.");

        System.out.println(
            "Hash Value: " + hashFunction(emp.id));
    }

    // Search employee
    static void searchEmployee(int id) {

        ArrayList<Employee> employees =
            readEmployees();

        for (Employee e : employees) {

            if (e.id == id) {

                System.out.println(
                    "Employee Found:");
                System.out.println(e);
                return;
            }
        }

        System.out.println(
            "Employee not found.");
    }

    // Update employee
    static void updateEmployee(
        int id,
        String name,
        String department,
        double salary) {

        ArrayList<Employee> employees =
            readEmployees();

        for (Employee e : employees) {

            if (e.id == id) {

```

```

        e.name = name;
        e.department = department;
        e.salary = salary;

        writeEmployees(employees);

        System.out.println(
            "Employee updated successfully.");
        return;
    }
}

System.out.println(
    "Employee not found.");
}

// Delete employee
static void deleteEmployee(int id) {

    ArrayList<Employee> employees =
        readEmployees();

    Iterator<Employee> it =
        employees.iterator();

    while (it.hasNext()) {

        Employee e = it.next();

        if (e.id == id) {

            it.remove();

            writeEmployees(employees);

            System.out.println(
                "Employee deleted successfully.");
            return;
        }
    }

    System.out.println(
        "Employee not found.");
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    while (true) {

        System.out.println(
            "\n--- EMPLOYEE MANAGEMENT ---");

        System.out.println("1. Add Employee");
        System.out.println("2. Search Employee");
        System.out.println("3. Update Employee");
        System.out.println("4. Delete Employee");
        System.out.println("5. Exit");
    }
}

```

```

System.out.print("Enter choice: ");
int choice = sc.nextInt();

switch (choice) {

    case 1:

        System.out.print(
            "Enter Employee ID: ");
        int id = sc.nextInt();

        sc.nextLine();

        System.out.print(
            "Enter Name: ");
        String name = sc.nextLine();

        System.out.print(
            "Enter Department: ");
        String dept = sc.nextLine();

        System.out.print(
            "Enter Salary: ");
        double salary = sc.nextDouble();

        addEmployee(
            new Employee(
                id, name, dept, salary));
        break;

    case 2:

        System.out.print(
            "Enter Employee ID: ");
        id = sc.nextInt();

        searchEmployee(id);
        break;

    case 3:

        System.out.print(
            "Enter Employee ID: ");
        id = sc.nextInt();

        sc.nextLine();

        System.out.print(
            "Enter New Name: ");
        name = sc.nextLine();

        System.out.print(
            "Enter New Department: ");
        dept = sc.nextLine();

        System.out.print(

```

```

        "Enter New Salary: ");
        salary = sc.nextDouble();

        updateEmployee(
            id, name, dept, salary);
        break;

    case 4:

        System.out.print(
            "Enter Employee ID: ");
        id = sc.nextInt();

        deleteEmployee(id);
        break;

    case 5:

        System.out.println(
            "Program terminated.");
        sc.close();
        return;

    default:

        System.out.println(
            "Invalid choice.");
    }
}
}
}

```

Input

--- EMPLOYEE MANAGEMENT ---

1. Add Employee
2. Search Employee
3. Update Employee
4. Delete Employee
5. Exit

Enter choice: 1
 Enter Employee ID: 101
 Enter Name: Ravi
 Enter Department: IT
 Enter Salary: 50000

Output

Employee added successfully.
 Hash Value: 1

Search Input

Enter choice: 2
 Enter Employee ID: 101

Search Output

Employee Found:
 ID: 101
 Name: Ravi
 Department: IT

Salary: 50000.0

Duplicate ID Output

```
Enter choice: 1
Enter Employee ID: 101
Enter Name: Arun
Enter Department: HR
Enter Salary: 45000
```

Error: Duplicate Employee ID.

Concepts Used

- Hashing
 - Hash function: `id % 10`
 - `FileInputStream`
 - `FileOutputStream`
 - `ObjectInputStream`
 - `ObjectOutputStream`
 - **Serialization**
 - `ArrayList`
 - Add/Search/Update/Delete
 - Exception handling
 - Duplicate ID validation
-

2. Bank Transaction File and Database System

Aim

To develop a banking application using **Java I/O, serialization, and JDBC** to maintain customer and transaction information and perform `INSERT`, `UPDATE`, `DELETE`, and `SELECT` database operations.

Database Setup

Create the database and table first:

```
CREATE DATABASE bankdb;
```

```
USE bankdb;
```

```
CREATE TABLE customer (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    balance DOUBLE
);
```

Java Program

```
import java.io.*;import java.sql.*;import java.util.*;
class Customer implements Serializable {

    private static final long serialVersionUID = 1L;

    int id;
    String name;
    double balance;
```

```

Customer(int id, String name, double balance) {

    this.id = id;
    this.name = name;
    this.balance = balance;
}

public String toString() {

    return "ID: " + id +
           ", Name: " + name +
           ", Balance: " + balance;
}
}

public class BankSystem {

    static final String URL =
        "jdbc:mysql://localhost:3306/bankdb";

    static final String USER = "root";
    static final String PASSWORD = "root";

    // Database connection
    static Connection getConnection()
        throws SQLException {

        return DriverManager.getConnection(
            URL, USER, PASSWORD);
    }

    // INSERT
    static void insertCustomer(
        int id,
        String name,
        double balance) {

        if (balance < 0) {

            System.out.println(
                "Invalid balance.");
            return;
        }

        String sql =
            "INSERT INTO customer VALUES (?, ?, ?)";

        try (Connection con = getConnection();
            PreparedStatement ps =
                con.prepareStatement(sql)) {

            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setDouble(3, balance);

            ps.executeUpdate();

            System.out.println(

```

```

        "Customer inserted successfully.");
    }

    catch (SQLException e) {

        System.out.println(
            "Database Error: " +
            e.getMessage());
    }
}

// SELECT
static void selectCustomers() {

    String sql =
        "SELECT * FROM customer";

    try (Connection con = getConnection();
        Statement st =
            con.createStatement();
        ResultSet rs =
            st.executeQuery(sql)) {

        System.out.println(
            "\n--- CUSTOMER RECORDS ---");

        while (rs.next()) {

            System.out.println(
                "ID: " + rs.getInt("id") +
                ", Name: " +
                rs.getString("name") +
                ", Balance: " +
                rs.getDouble("balance"));

        }
    }

    catch (SQLException e) {

        System.out.println(
            "Database Error: " +
            e.getMessage());
    }
}

// UPDATE
static void updateBalance(
    int id,
    double amount) {

    if (amount < 0) {

        System.out.println(
            "Invalid amount.");
        return;
    }
}

```

```

String sql =
    "UPDATE customer " +
    "SET balance = balance + ? " +
    "WHERE id = ?";

try (Connection con = getConnection();
    PreparedStatement ps =
        con.prepareStatement(sql)) {

    ps.setDouble(1, amount);
    ps.setInt(2, id);

    int rows =
        ps.executeUpdate();

    if (rows > 0)
        System.out.println(
            "Balance updated successfully.");
    else
        System.out.println(
            "Customer not found.");

}

catch (SQLException e) {

    System.out.println(
        "Database Error: " +
        e.getMessage());

}

}

// DELETE
static void deleteCustomer(int id) {

    String sql =
        "DELETE FROM customer WHERE id = ?";

    try (Connection con = getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql)) {

        ps.setInt(1, id);

        int rows =
            ps.executeUpdate();

        if (rows > 0)
            System.out.println(
                "Customer deleted successfully.");
        else
            System.out.println(
                "Customer not found.");

    }

    catch (SQLException e) {

```

```

        System.out.println(
            "Database Error: " +
            e.getMessage());
    }
}

// Transaction log using character stream
static void writeLog(String message) {

    try (FileWriter fw =
        new FileWriter(
            "transactions.txt", true);
        BufferedWriter bw =
            new BufferedWriter(fw)) {

        bw.write(message);
        bw.newLine();

    }

    catch (IOException e) {

        System.out.println(
            "I/O Error: " +
            e.getMessage());
    }
}

// Serialize customer object
static void backupCustomer(
    Customer customer) {

    try (ObjectOutputStream out =
        new ObjectOutputStream(
            new FileOutputStream(
                "customer_backup.dat"))) {

        out.writeObject(customer);

        System.out.println(
            "Customer backup created.");

    }

    catch (IOException e) {

        System.out.println(
            "Serialization Error: " +
            e.getMessage());
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    try {

        System.out.print(

```

```

        "Enter Customer ID: ");
int id = sc.nextInt();

sc.nextLine();

System.out.print(
    "Enter Customer Name: ");
String name = sc.nextLine();

System.out.print(
    "Enter Initial Balance: ");
double balance = sc.nextDouble();

if (balance < 0) {

    System.out.println(
        "Invalid balance.");
    return;
}

// INSERT
insertCustomer(
    id, name, balance);

writeLog(
    "Customer created: ID=" +
    id);

// BACKUP
Customer c =
    new Customer(
        id, name, balance);

backupCustomer(c);

// SELECT
selectCustomers();

// UPDATE
System.out.print(
    "\nEnter deposit amount: ");

double amount =
    sc.nextDouble();

if (amount <= 0) {

    System.out.println(
        "Invalid transaction amount.");
}
else {

    updateBalance(id, amount);

    writeLog(
        "Deposit: ID=" + id +
        ", Amount=" + amount);
}

```

```

    }

    // SELECT after update
    selectCustomers();

    // DELETE
    System.out.print(
        "\nEnter customer ID to delete: ");

    int deleteId =
        sc.nextInt();

    deleteCustomer(deleteId);

    writeLog(
        "Customer deleted: ID=" +
        deleteId);

    }

    finally {

        sc.close();

    }

}

```

Input

```

Enter Customer ID: 101
Enter Customer Name: Ravi
Enter Initial Balance: 5000

Enter deposit amount: 2000

Enter customer ID to delete: 101

```

Output

```

Customer inserted successfully.
Customer backup created.

--- CUSTOMER RECORDS ---
ID: 101, Name: Ravi, Balance: 5000.0

Balance updated successfully.

--- CUSTOMER RECORDS ---
ID: 101, Name: Ravi, Balance: 7000.0

Customer deleted successfully.

```

Files Created

transactions.txt
 customer_backup.dat
 transactions.txt stores transaction logs using **character stream I/O**, while
 customer_backup.dat stores the serialized Customer object.

JDBC Operations Used

Operation	JDBC method
INSERT	PreparedStatement.executeUpdate()

Operation	JDBC method
SELECT	Statement.executeQuery()
UPDATE	PreparedStatement.executeUpdate()